

8

Handling files

This chapter demonstrates how to read and write files with PHP and how to deal with errors in scripts.

Reading files

Reading lines

Reading characters

Writing a file

Appending text

Handling errors

Catching exceptions

Summary

Reading files



The ability to read and write files located on the web server is useful to log information and to provide page content. In order to process files, a “filestream” object must first be created using the PHP **fopen()** function. This requires two arguments to specify the name or path of the file and a file “mode”. The file mode determines the nature of the operation to be performed and places the file pointer at a suitable position within the file:

File mode	Operation
r	Open an existing file to read File pointer starts at the beginning of the file
w	Discard all contents of an existing file or create a new file if none exists File pointer starts at the beginning of the file
a	Open an existing file and preserve all contents File pointer starts at the end of the file
r+	Open a file to read from or write to File pointer starts at the beginning of the file
w+	Open a file to write to or read from Discard all contents of an existing file or create a new file if none exists File pointer starts at the beginning of the file
a+	Open a file to read from or write to and preserve all contents File pointer starts at the end of the file



The file mode specifier must appear within quote marks when supplied as an argument to the **fopen()** function.

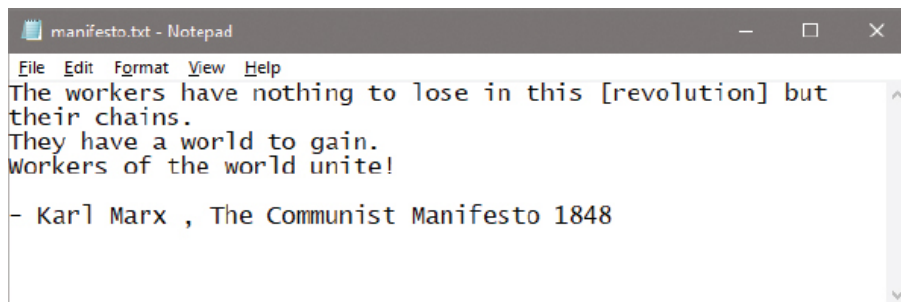
It is good practice to always provide an alternative with the **die()** function in case the attempt to create a filestream fails.



Specify an explanatory message as an argument to the **die()** function.

When a filestream is created successfully, the file can be read by specifying the filestream and byte size as arguments to the **fread()** function. If the whole file is to be read it is convenient to use the **fsize()** function to specify the byte size of the file to be read.

Upon completion of any file operation, the filestream must be destroyed by specifying it as an argument to the **fclose()** function.



manifesto.txt

1. Begin a PHP script with statements that attempt to open a plain text file for reading, or exit the script upon failure

<?php

\$filestream = fopen('manifesto.txt' , 'r')

or die('Unable to open file!') ;



read.php

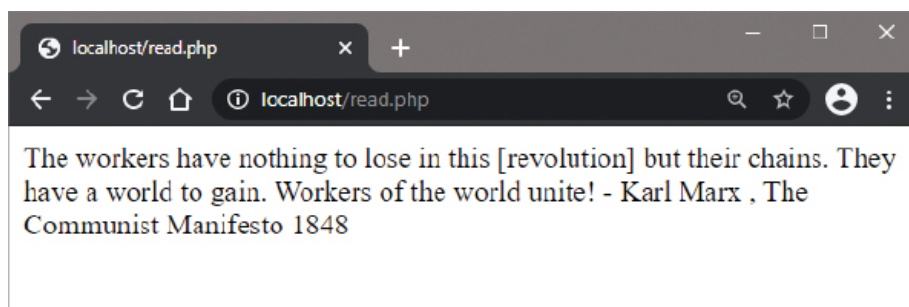
2 Next, add a statement to read the text file upon success and display its entire contents

```
echo fread( $filestream , filesize( 'manifesto.txt' ) ) ;
```

3 Now, remember to close the filestream

```
fclose( $filestream ) ;
```

4 Save the document in your web server's **/htdocs** directory as **read.php**, alongside the plain text file to be read, then open the page via HTTP to see the text file contents displayed in the browser window



Unlike a PHP block within an HTML document, a purely PHP script needs no closing `?>` tag.

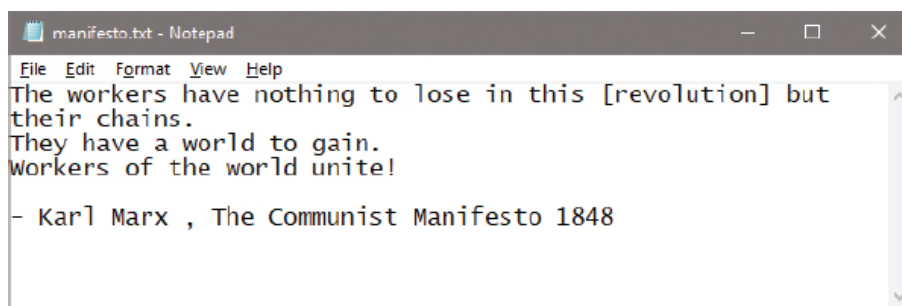


Notice that the output here does not maintain line endings – but the example here does.

Reading lines



Once a “filestream” object is created, its contents can be read one line at a time by specifying the filestream as an argument to the PHP **fgets()** function. To read every line, this function can be called repeatedly in a loop. The **feof()** function can determine when the end of the file is reached and can end the loop:



The **feof()** function returns true when the end of the file is reached.

1. Begin a PHP script with statements that attempt to open a plain text file for reading, or exit the script upon failure

```
<?php
```

```
$filestream = fopen( 'manifesto.txt' , 'r' )
```

```
or die( 'Unable to open file!' ) ;
```



readlines.php

2. Next, add a loop to read the text file line-by-line upon success and display its entire contents

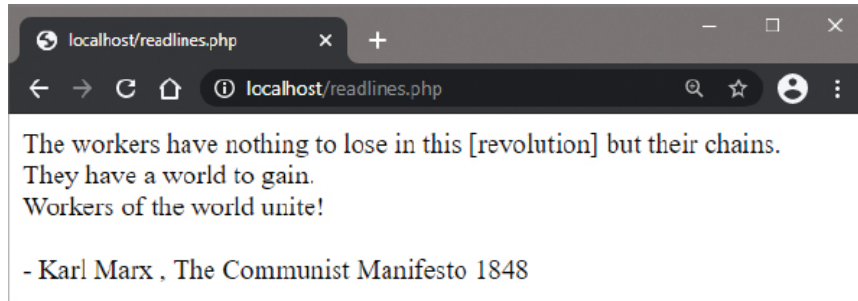
```
while( !feof( $filestream ) )
```

```
{ echo fgets( $filestream ).'<br>' ; }
```

3 Now, remember to close the filestream

```
fclose( $filestream );
```

4 Save the document in your web server's **/htdocs** directory as **readlines.php**, alongside the plain text file to be read, then open the page via HTTP to see the text file lines

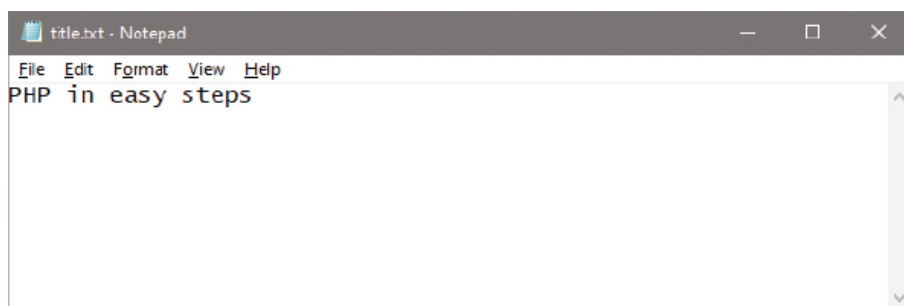


The next to last line is not actually empty – it contains an invisible newline character.

Reading characters



Once a “filestream” object is created, its contents can be read one character at a time by specifying the filestream as an argument to the PHP **fgetc()** function. To read every character, this function can be called repeatedly in a loop. The **fgetc()** function will return false when the end of the file is reached and can end the loop:



title.txt

- 1 Begin a PHP script with statements that attempt to open a plain text file for reading, or exit the script upon failure

```
<?php
```

```
$filestream = fopen( 'title.txt' , 'r' )
```

```
or die( 'Unable to open file!' ) ;
```



readchars.php

- 2 Next, add a loop to read the text file character-by-character upon success and display its entire contents

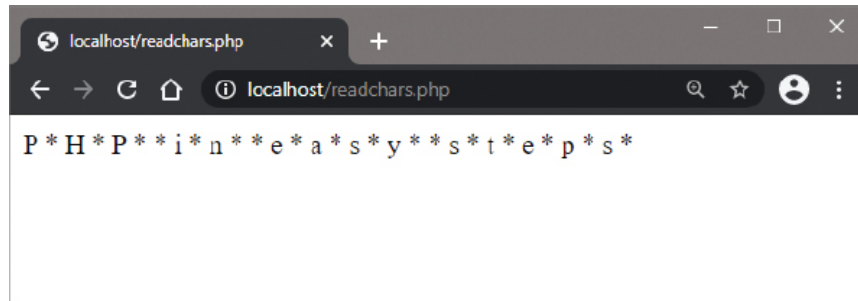
```
while( $char = fgetc( $filestream ) )
```

```
{ echo $char. ' * ' ; }
```

3 Now, remember to close the filestream

```
fclose( $filestream );
```

4 Save the document in your web server's **/htdocs** directory as **readchars.php**, alongside the plain text file to be read, then open the page via HTTP to see the file characters



The **fgetc()** function returns false when there are no more characters to get from the stream.

Writing a file



Once a “filestream” object has been created specifying a write mode, a text file can be created. Text can be written into the file by specifying the filestream and a file name to the PHP **fwrite()** function. To format the text, non-printing invisible characters can also be written to create line breaks and tab spacing:

Character	Description
<code>\n</code>	Newline Move to a new line down
<code>\r</code>	Carriage return Move to the beginning of the line
<code>\t</code>	Tab Move along the line to the next tab stop

1 Begin a PHP script with a statement to assign text, including non-printing characters for formatting

```
<?php
```

```
$poem = “\r\n\tI never saw a man who looked” ;
```

```
$poem .= “\r\n\tWith such a wistful eye” ;
```

```
$poem .= “\r\n\tUpon that little tent of blue” ;
```

```
$poem .= “\r\n\tWhich prisoners call the sky” ;
```



write.php

2 Next, assign a name for the file to be written

```
$filename = ‘poem.txt’ ;
```

3 Attempt to create a plain text file for writing, or exit the script upon failure

```
$filestream = fopen( $filename , 'w' )
```

```
or die( 'Unable to open file!' ) ;
```

4 Write the file and display a confirmation of how many bytes have been written in total

```
$num = fwrite( $filestream , $poem ) ;
```

```
if( $num )
```

```
{
```

```
    echo “$num bytes written to $filename” ;
```

```
}
```

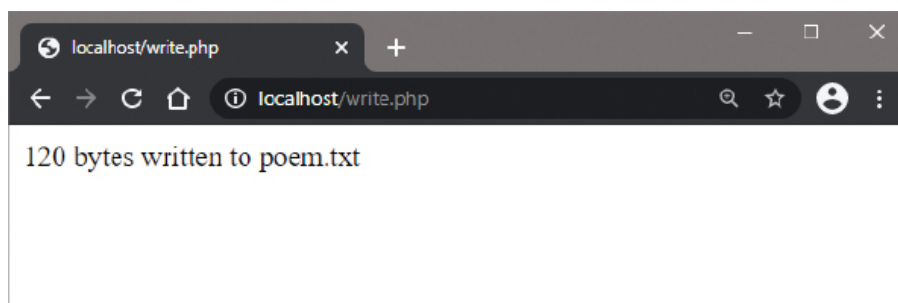
5 Now, remember to close the filestream

```
fclose( $filestream ) ;
```

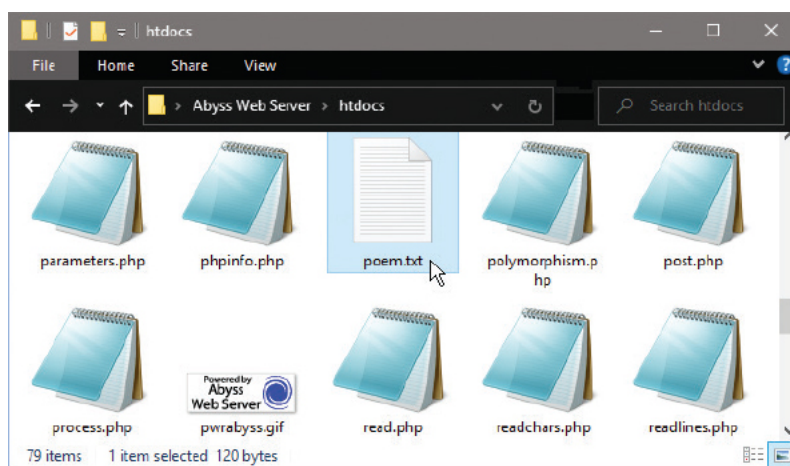


Strings containing non-printing characters must be enclosed within double quotes so that PHP can interpret them.

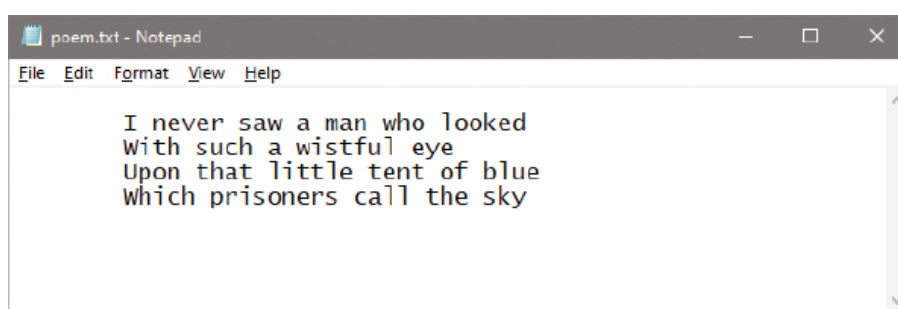
6 Save the document in your web server's **/htdocs** directory as **write.php** then open the page via HTTP to see confirmation that the text file has been written



7 Use File Explorer to navigate to your web server's **/htdocs** directory and see the file created by the PHP script



8 Open the file to see the text written by the PHP script



The folder where a file is to be written must have write permission.



You can run this example repeatedly and each time it will erase the content then write it again.

Appending text



Once a “filestream” object has been created specifying an append mode, a text file can be opened to add text at its end. Text can be appended to the file by specifying the filestream and a file name to the PHP **fwrite()** function. The process is identical to writing a file, except that the file pointer is positioned at the end of existing content before the text is written into the file:

- 1 Begin a PHP script with a statement to assign text, including non-printing characters for formatting

```
<?php
```

```
$info = “\r\n\r\n\tThe Ballad of Reading Gaol” ;
```

```
$info .= “\r\n\t\t\tOscar Wilde 1898” ;
```



append.php

- 2 Next, assign a name for the file to be written into

```
$filename = ‘poem.txt’ ;
```

- 3 Attempt to open the plain text file for appending, or exit the script upon failure

```
$filestream = fopen( $filename , ‘a’ )
```

```
or die( ‘Unable to open file!’ ) ;
```

- 4 Write the text and display a confirmation of how many bytes have been appended in total

```
$num = fwrite( $filestream , $info ) ;
```

```
if( $num )
```

```
{
```

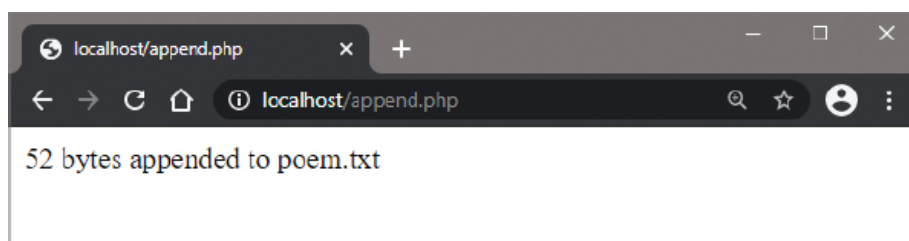
```
    echo “$num bytes appended to $filename” ;
```

```
}
```

5 Now, remember to close the filestream

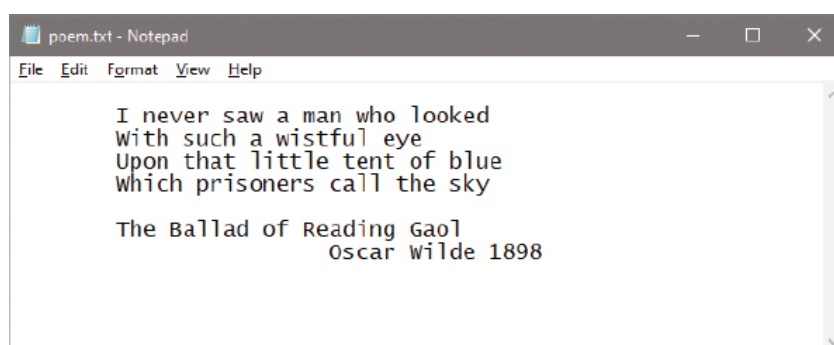
```
fclose( $filestream ) ;
```

6 Save the document in your web server's **/htdocs** directory as **append.php** then open the page via HTTP to see confirmation that the text has been appended to the file



Remember that non-printing characters must be enclosed in double quotes for interpretation.

7 Navigate to your web server's **/htdocs** directory and open the file to see the text appended by the PHP script



Overwriting text

It is important to recognize that when you write to an existing file in write mode it will overwrite any existing content:

1 Begin a PHP script by attempting to open an existing text file for writing, or exit the script upon failure

```
<?php
```

```
$filestream = fopen( 'poem.txt' , 'w' )
```

```
or die( 'Unable to open file!' ) ;
```



overwrite.php

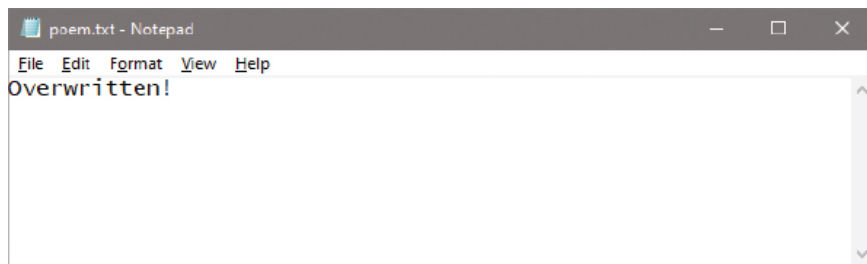
2 Upon success, write new text into the file

```
fwrite( $filestream , 'Overwritten!' );
```

3 Now, remember to close the filestream

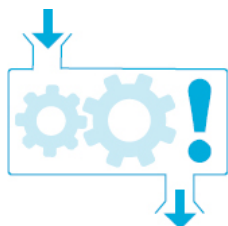
```
fclose( $filestream );
```

4 Save the document in your web server's **/htdocs** directory as **overwrite.php** then open the page via HTTP and reopen the text file to see its contents overwritten

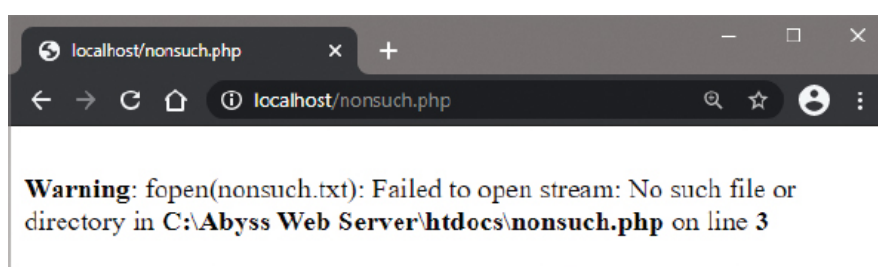


Be careful when writing to files as it is easy to accidentally overwrite.

Handling errors



You can anticipate possible errors and terminate a running script when they occur using the **die()** function or the **exit()** function. Without this contingency, the default PHP error handler provides a useful but unsightly message when an error occurs. For example, **fopen('nonsuch.txt' , 'r')** to open a non-existing file:



The **die()** and **exit()** functions perform identical operations in PHP, but their terminology can have different appeals. For some, **die()** possibly seems more appropriate to quit when an error is detected, and **exit()** may seem more suitable to quit in other scenarios.

It is not always desirable to terminate a script when an error occurs, but you can create a custom error handler to provide a more user-friendly message than that provided by the default PHP error handler. A custom error handler is simply a function with this syntax:

function error_handler(*level* , *message* , *file* , *line* , *context*)

Only the first two parameters are required, the rest are optional. The first of these is a numerical value, representing a constant, indicating the severity of the error that has occurred:

Level	Constant	Description

2	E_WARNING	Non-fatal runtime error
8	E_NOTICE	Non-fatal possible error
256	E_USER_ERROR	Fatal user-generated error
512	E_USER_WARNING	Non-fatal user-generated error



The basic error levels are listed in this table but the PHP manual lists many more ([php.net/ manual](http://php.net/manual)).

A custom error handler function is nominated to handle errors by specifying its function name as the argument to the built-in PHP **set_error_handler()** function.

The error handler is called when an error occurs, or can be called explicitly using the **trigger_error()** function to specify an error message as its argument.

1 Begin a PHP script by defining a custom error handler function to report an error level and error message

```
<?php
function error_handler( $level , $message )
{
    switch( $level )
    {
        case 2 : $str = 'Warning!' ; break ;
        case 8 : $str = 'Notice!' ; break ;
        default : $str = 'Error!' ;
    }
    echo "<b>$str</b><br>$message<hr>";
}
```



error.php

2 Next, nominate the function to handle errors

```
set_error_handler( 'error_handler' );
```

3 Now, for some mischief – attempt to access an uninitialized variable and try to open a non-existing file

```
echo( $var );
```

```
$file = fopen( 'nonsuch.txt' , 'r' );
```

4 Explicitly trigger the custom error handler

```
$number = 2 ;
```

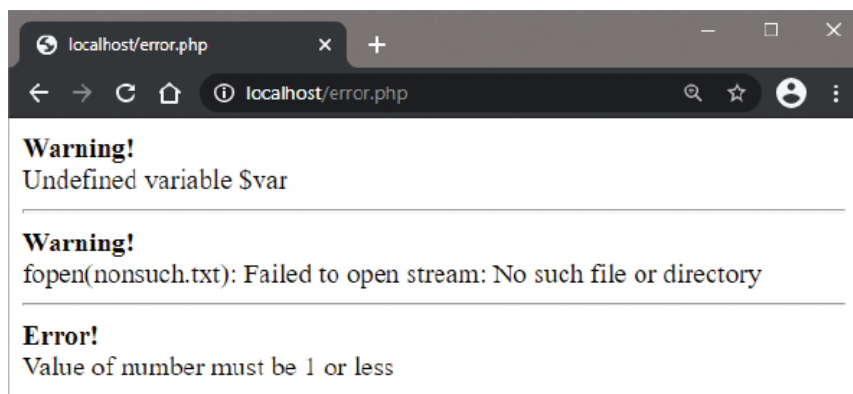
```
if( $number >= 1 )
```

```
{
```

```
    trigger_error( 'Value of number must be 1 or less' );
```

```
}
```

5 Save the document in your web server's **/htdocs** directory as **error.php** then open the page via HTTP to see the error levels and appropriate messages

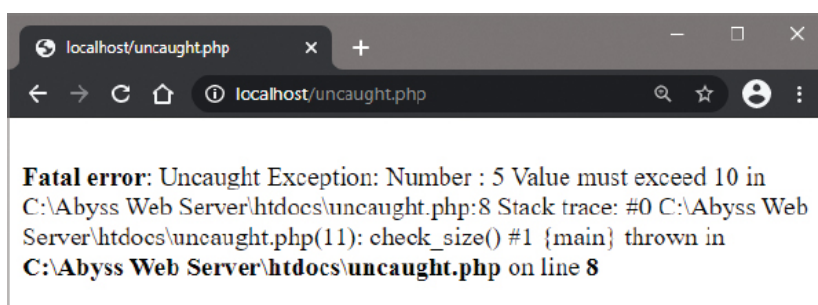


Notice that the script continues after reporting each error; it does not as terminate on error.

Catching exceptions



We can choose how a script should proceed when a specific error condition occurs. This error condition is called an “exception”. When an exception occurs it is said to be “thrown”, and code handling the exception is said to be its “catch”. If an exception is not caught, PHP will treat it as a fatal error and issue an unsightly “Uncaught Exception” message – something like this:



Each **try** block must have an associated **catch** block, otherwise a fatal Uncaught Exception error will occur.

The key to handling exceptions is to anticipate the error by trying to execute an action within a **try** code block, and determine how the script should proceed in an associated **catch** code block. The catching block is passed an **Exception** class object that has **getMessage()**, **getFile()** and **getLine()** methods to retrieve details.

An instance of the **Exception** class can be created using the **new** keyword, and can be explicitly thrown using the **throw** keyword.

A custom exception handler can also be created as a derived class of the **Exception** class, to provide your own custom methods:

1 Begin a PHP script by defining a function that will throw an exception when a conditional test fails

```
<?php
function check_size( $num )
{
    if( $num < 10 )
    { throw new
    Exception( "Number : $num<br>Value must exceed 10" ) ; }
}
```



exception.php

2 Anticipate a specific error and report the exception

```
try { check_size( 5 ) ; }
catch( Exception $e )
{
    echo '<b>Size Exception!</b><br>'.
    $e->getMessage().'<hr>' ;
}
```

3 Next, create a custom exception handler class

```
class CustomException extends Exception
{
    public function get_details()
    {
        $details = 'File : '.$this->getFile().
        '<br>Line : '.$this->getLine().
        '<br>'.$this->getMessage() ;
        return $details ;
    }
}
```

4 Now, add another function that will throw an exception when a conditional test fails

```
function check_parity( $num )
```

```

{
    if( $num % 2 != 0 )
    {
        throw new CustomException( "Number :
                                   $num<br>Value must be even" );
    }
}

```

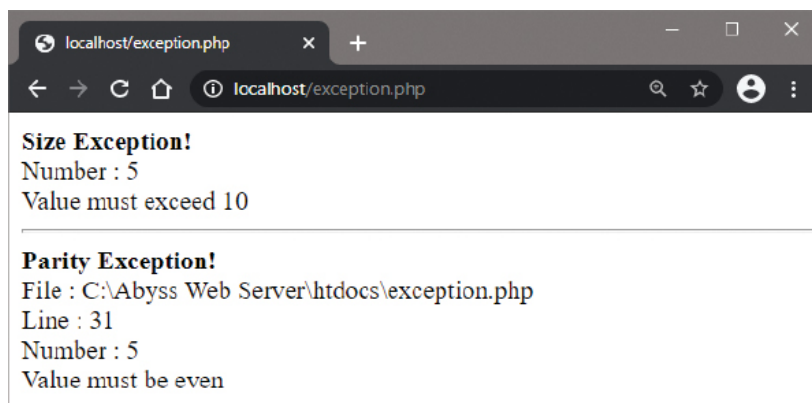
5 Finally, anticipate a specific error and report the exception

```

try { check_parity( 5 ); }
catch( CustomException $e )
{   echo '<b>Parity Exception!</b><br>'.
        $e->get_details().'<br>' ;
}

```

6 Save the document in your web server's **/htdocs** directory as **exception.php** then open the page via HTTP to see the exceptions caught and appropriate message details



For more on derived classes and inheritance see [here](#).



A **try** block can contain multiple **throw** statements to anticipate multiple error conditions, and can be followed by multiple **catch** blocks to handle those exceptions.

Summary

- The **fopen()** function is used to create a filestream object and requires two arguments to specify a file name and a file mode.
- It is good practice to provide an alternative with the **die()** function in case an attempt to create a filestream fails.
- The **fread()** function is used to read a filestream created using the 'r' file mode, and the **filesize()** function can be used to specify the byte size of the file.
- Upon completion of any file operation, the filestream must be destroyed by calling the **fclose()** function.
- The **fgets()** function can be used to read a file line-by-line, and the **feof()** function can recognize the end of a file.
- The **fgetc()** function can be used to read a file character-by-character and will return false at the end of the file.
- Text can be written into a file by specifying the filestream and the file name as arguments to the **fwrite()** function.
- The **fwrite()** function will overwrite existing text using the 'w' file mode but will append text when using the 'a' file mode.
- The name of a function can be specified as the argument to the **set_error_handler()** function to nominate it as a custom error handler.
- The error handler function is called when an error occurs or can be called explicitly using the **trigger_error()** function.
- An exception is thrown when a specific error condition occurs.
- If an exception is not caught, PHP will treat it as a fatal error and issue an Uncaught Exception message.
- A **try** code block is used to anticipate a specific error, and its associated **catch** block is used to determine how the script should proceed when that error occurs.
- An **Exception** class object has **getMessage()**, **getFile()** and **getLine()** methods that can retrieve exception details.

- A custom exception handler can be created as a derived class of the **Exception** class to provide custom exception methods.